BIOT 6900 · AI-POWERED BIOTECH WORKFLOWS

Module 5 Lab Guide, Part 1: Variant Interpretation from First Principles

Week 5 · the toy model · instructor-led, not graded · Fall 2026

Notebook: `BIOT6900_Module5_Part1_Toy.ipynb` Runtime: about 30 minutes No data files, no downloads, no network

What this lab is

In the lecture you met CADD, REVEL, SpliceAI, AlphaMissense and the AlphaGenome Atlas. Each is a large model whose internals you cannot inspect. The risk, once you start using them, is that a number becomes an oracle: you have no way to ask why it came out as it did, so the only thing left to do with it is believe it.

This lab removes that risk by having you build miniature versions of four of those tools, about forty lines of code each, on a 90-base toy gene you can read in full. By the end you will have watched each miniature go completely blind on a variant that another one catches easily.

Step	You build	The real tool it stands in for
1	A consequence caller	Ensembl VEP
2	A conservation scorer	SIFT, and the dominant signal inside CADD and REVEL
3	A splice-site model producing delta scores	SpliceAI
4	A transcription-factor binding model	AlphaGenome, in miniature
5	All four on one variant panel	the coverage matrix from the lecture

The one idea to carry through

A predictor is a function from sequence to a number. It can only see what its input and its training signal let it see. Every blind spot you find in this notebook is a real blind spot of the real tool it stands in for.

Maps to course outcomes: LO1 (apply industry-standard tools), LO3 (interpret results in biological context), LO4 (evaluate model performance and limitations).

Before you start

1 · Your environment

Use the same biot6900 conda environment from Module 1. This notebook needs only numpy, pandas and matplotlib, all of which are already in it. There is nothing to install and nothing to download.

If your environment is broken, come to office hours before class rather than losing the session to setup.

2 · What you do not need

- No data files. The gene is 90 characters, written into the notebook.

- No internet connection. Nothing is fetched.
- No API keys, no model weights, no GPU.

This is deliberate. Part 1 is the only notebook in the module with no external dependency at all, so nobody is blocked while we work through the mechanics.

The toy gene

Real variant interpretation fails on bookkeeping more often than on modelling, so the lab starts with a gene whose every base is visible. It has everything a real gene has, at roughly 1/1000 scale.

```
promoter  g.1-24   GGCATTAC TGACGTCA GGTTCCAA      TF motif at g.9-16
5'UTR     g.25-30  ACACCC
exon 1    g.31-60  ATG GCA GGC AAG TTA AAG CGT GAC CTG CAA
intron    g.61-78  GT GAGTACCTTTTCCC AG            canonical GT ... AG
exon 2    g.79-90  GGC TAC TTG TAA

spliced coding sequence   ATGGCAGGCAAGTTAAAGCGTGACCTGCAAGGCTACTTGTAA
protein                   MAGKLKRDLQGYL*
```

Thirteen residues and a stop codon. Small enough that you can check by hand any claim the code makes. Positions are 1-based and inclusive, matching HGVS convention.

The eight variants, and why each one is there

The panel is not random. Each variant is placed so that exactly one predictor should notice it, which is how the blind spots become visible.

Variant	Region	What it is designed to show
`g.10G>T`	promoter	**Destroys the transcription-factor motif. Regulatory effect, no protein change at all.**
`g.16A>G`	promoter	**Weakens the same motif partially. Regulatory effects are graded, not binary.**
`g.36A>G`	exon 1	**Synonymous and genuinely harmless. The negative control.**
`g.39C>T`	exon 1	**Synonymous, and creates a cryptic splice donor. The punchline of the lab.**
`g.46A>T`	exon 1	**Nonsense at residue 6 of 13. Truncation that the consequence caller alone catches.**
`g.53A>G`	exon 1	**Missense D8G, at a column invariant across all ten species in the alignment.**
`g.61G>A`	intron	**Destroys the canonical GT. The easy splice case.**
`g.70T>C`	intron	**Deep intronic, far from any site. Should be silent everywhere.**
`g.82T>C`	exon 2	**Missense Y12H, at the most variable column in the protein.**

Do this before running anything

Look at the nine variants above and predict which ones a missense-only predictor such as REVEL would be able to score at all. Write your answer down. Step 5 checks it.

Walking through the five steps

Step 1 · The consequence caller

What it does. Looks up which region the position falls in. If promoter or UTR it stops there. If intronic it checks whether the variant hits the first two or last two intronic bases. If exonic it splices the two exons together, translates both alleles, and compares the proteins.

What you should see. Eight consequence terms. Two of them read `synonymous_variant`: g.36A>G and g.39C>T.

The point. A consequence is a derived property, not an intrinsic one. The transcript model is hard-coded in a single line of this notebook; change which exons you splice and the same variant gets a different consequence. That is the transcript-choice problem from the lecture, in miniature. Notice also that there is no machine learning in this step at all: a consequence term is bookkeeping.

Step 2 · The conservation scorer

What it does. Takes a ten-species alignment of the 13-residue protein and scores a substitution as a log ratio: how often the alternative amino acid appears in that column, divided by how often the reference does. A pseudocount of 0.1 stops a count of zero producing negative infinity.

What you should see. Two scores for the same class of substitution, seven log units apart:

```
D8G   (column 8 is aspartate in all ten species)        -6.66
Y12H  (column 12 holds nine different residues)         -0.93
```

The point, and its limitation in the same breath. That difference is real signal and it is why conservation-based predictors work at all. But notice what produced it: ten sequences. Our estimate of "invariant" rests on a sample of ten. A column can look invariant simply because it is under-sampled, and a gene under weak purifying selection shows shallow apparent conservation everywhere. Pharmacogenes are exactly such genes, which is why these predictors are poorly calibrated on them.

Step 3 · The splice model and what a delta score is

This is the step worth slowing down on, because delta scores are the most frequently misread numbers in the module. The code does four things:

- Learn the donor consensus. Count bases at each of nine positions across twelve donor sequences and store log-odds against background. That is a position weight matrix.
- Score every window in the gene. Any 9-mer without GT at positions 4 and 5 gets probability zero: no GT, no donor site.
- Score every window again on the mutated sequence.
- Subtract. The largest increase is the donor gain score; the largest decrease is the donor loss score.

What you should see.

```
real donor,  window g.58:  CAAGTGAGT   p = 0.954
latent site, window g.35:  CAGGCAAGT   p = 0.000

g.39C>T  synonymous_variant     gain 1.000 @g.35   loss 0.003
```

```
g.61G>A  splice_donor_variant   gain 0.000        loss 0.954
g.53A>G  missense_variant D8G   gain 0.000        loss 0.002
```

Why g.39C>T works. The window starting at g.35 reads CAGGCAAGT. A single C to T at position 39 makes it CAGGTAAGT, which is the donor consensus exactly. In the reading frame, codon GGC becomes GGT: glycine either way, so the protein is untouched.

The point. A variant that changes no amino acid has created a splice donor as strong as the real one, 23 bases upstream, which would truncate the first exon. Step 1 called it silent and Step 2 could not score it at all. Compare the numbers against the thresholds from the lecture: 0.2 for high sensitivity, 0.5 for a balanced call, 0.8 for high precision.

> **What a delta score is a claim about**
>
> RNA. It predicts a transcript, not a disease. The next step after a score like this is an RNA assay, not a diagnosis. Nothing in the calculation refers to a patient, a phenotype or a gene-disease relationship.

Step 4 · The regulatory model

What it does. The same shape of computation as Step 3, with one transcription factor and one promoter. Score the motif with a PWM, convert the score to an occupancy between 0 and 1, and turn occupancy into a predicted expression level.

```
expression_fold  =  (BASAL + occupancy_alt) / (BASAL + occupancy_ref)

BASAL is the transcription that happens with no factor bound, so losing
the site reduces expression rather than abolishing it.
```

What you should see.

Variant	Expression	Why
`g.10G>T`	x0.28	**Hits a high-information base of the motif. Occupancy falls by 0.71.**
`g.16A>G`	x0.62	**Hits a tolerant base. Partial loss: regulatory effects are graded, not switch-like.**
`g.3C>A`	x1.00	**Inside the promoter but outside the motif. Correctly silent.**

The point. There is no protein anywhere in this calculation, which is the entire reason the step exists. g.10G>T knocks expression to 28% of normal and Steps 1 to 3 are all silent on it. This is the VKORC1 case from the lecture: rs9923231 is a promoter variant that lowers VKORC1 expression and changes warfarin dosing in routine practice, and every missense predictor in this course returns nothing for it.

Step 5 · All four predictors on one panel

The final cell runs everything on nine variants at once. Read the shape of the table rather than any single number.

Variant	Consequence	conserv.	donor_gain	donor_loss	expr_fold
g.10G>T	promoter_variant	—	0.000	0.000	0.28
g.16A>G	promoter_variant	—	0.000	0.011	0.62
g.36A>G	synonymous	—	0.000	0.000	1.00
g.39C>T	synonymous	—	1.000	0.003	1.00
g.46A>T	stop_gained	—	0.000	0.002	1.00
g.53A>G	missense D8G	-6.66	0.000	0.002	1.00
g.61G>A	splice_donor	—	0.000	0.954	1.00
g.70T>C	intron_variant	—	0.000	0.000	1.00
g.82T>C	missense Y12H	-0.93	0.000	0.000	1.00

Read the table by its blanks, not its numbers

Seven of nine variants have no conservation score at all. It is a missense-only tool, exactly like REVEL and AlphaMissense.

The splice column fires once, on a variant the protein-level tools called silent. The regulatory column fires twice, on variants no other column can see.

A dash means could not score. A zero means scored, and found nothing. Those are not the same thing, and conflating them is the most common error in this kind of analysis.

Three of these nine would be missed entirely by a missense-only pipeline, and nothing in the output would have warned you.

What maps to what

The real tools differ from these miniatures in degree, not in kind: deeper training data, learned features instead of hand-built matrices, and genuine calibration. The failure modes you just watched are the real ones.

Your toy function	The real tool	What the real one adds	What it still does not fix
consequence()	Ensembl VEP	Every transcript and gene, real annotation	Still depends on which transcript you pick
conservation_score()	SIFT, REVEL, CADD	Thousands of species, learned weights	Still weak where selection is weak
donor_prob()	SpliceAI, Pangolin	A deep network over 10 kb of context	Still predicts RNA, not disease
tf_occupancy()	AlphaGenome Atlas	Many tissues, all 9 billion SNVs	Still not validated for clinical use

Troubleshooting

Symptom	Likely cause and fix
Every cell looks like one long line	**Your copy of the notebook was saved with broken line endings. Download it again from Canvas.**
ModuleNotFoundError: matplotlib	**You are not in the biot6900 environment. Activate it and restart the kernel.**
AssertionError on len(GENE) == 90	**You edited the GENE string. Re-download, or count your bases: the five segments are 24, 6, 30, 18 and 12.**
AssertionError: alt allele equals the reference base	**You asked for a variant whose alternative matches what is already there. Check the base at that position.**
KeyError from the CODON table	**The spliced sequence length is no longer a multiple of three. You changed an exon boundary.**
Donor scores are all zero	**Expected for any window with no GT at positions 4 and 5. If the real site at g.58 also reads zero, you changed the intron start.**
Numbers differ from this guide	**Run Kernel → Restart & Run All. The predictors depend on cells defined earlier, so running out of order silently changes results.**

What this lab deliberately leaves out

- **No real evidence.** No population frequencies, no prior clinical assertions, no functional assay data. Every number came from sequence alone.
- **No calibration.** The scores are raw. Turning a number into an evidence strength requires likelihood ratios estimated against curated variant sets, which needs real labels.
- **No ground truth.** Nothing here can be checked, so nothing here can be shown to be miscalibrated, circular or overconfident.

What comes next

Part 2 runs the real tools on real ClinVar variants in CYP2C19, CYP2C9, VKORC1, DPYD and TPMT, with ClinVar assertions as a reference standard. All three gaps above get filled, and then you examine how trustworthy that reference standard actually is. Keep this notebook: the reading habits transfer even though the code does not.